Cloud & Application (In)Security

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

Learning Objectives

- Define the CIA triad (confidentiality, integrity, availability) and explain how each pillar applies to cloud-hosted applications
- Identify common cloud security threats including traffic eavesdropping, man-in-the-middle attacks, denial of service, and insufficient authorization
- Explain the shared responsibility model and articulate which security tasks fall on the cloud provider versus the customer for IaaS, PaaS, and SaaS
- Compare symmetric and asymmetric encryption and describe how TLS uses both during a connection handshake
- Differentiate encryption at rest from encryption in transit and map each to specific AWS, GCP, and Azure services
- Use cloud key management and secrets management services to avoid hardcoded credentials in application code
- Describe common application-layer attacks (SQL injection, XSS, CSRF) and their defenses
- Identify cloud-native security services for DDoS protection, web application firewalls, threat detection, and audit logging across AWS, GCP, and Azure
- Apply the principle of least privilege when configuring IAM policies, security groups, and firewall rules

The CIA Triad

The Cost of Getting Security Wrong

- In 2019, a former employee exploited a misconfigured AWS WAF to access Capital One's S3 buckets, exposing over 100 million customer records.
  - The breach cost Capital One over $300 million in fines, settlements, and remediation.
  - The root cause was not a flaw in AWS — it was a misconfiguration by the customer.
  - This is exactly the kind of mistake that happens when engineers do not understand the shared responsibility model.
- Security is not someone else's problem. If you deploy code to the cloud, security is your problem.

The CIA Triad

Every security discussion starts with three fundamental properties that a system must protect:

- **Confidentiality** — Only authorized parties can access data. When a cloud consumer sends a message to a cloud service, was that message seen by anyone unauthorized along the way? Encryption, access controls, and network isolation all protect confidentiality.
- **Integrity** — Data has not been altered in an unauthorized way. When a message arrives at a cloud service, does it match exactly what the sender transmitted? Hashing, digital signatures, and checksums protect integrity.
- **Availability** — The service is accessible when needed. For a cloud service, the expectation is 24/7/365 uptime. DDoS attacks, resource exhaustion, and infrastructure failures all threaten availability.
- **Authenticity** (sometimes called the fourth pillar) — Is the data actually coming from who it claims to be from? Spoofed emails, forged API requests, and man-in-the-middle attacks all compromise authenticity. Digital certificates and signatures address this.

```
              Confidentiality
                   /\
                  /  \
                 /    \
                / CIA  \
               /  Triad \
              /          \
             /____________\
  Integrity               Availability
```

Why the CIA Triad Matters in the Cloud

- In traditional on-premises environments, the organization controls every layer from the physical server room to the application code.
- In the cloud, responsibility is split between you and the provider.
- Every decision you make — which encryption key to use, which ports to open, which IAM policy to attach — maps back to protecting one or more of these three properties.

Security Vocabulary

Vocabulary

- **Threat -** A threat is a possible danger that might exploit a vulnerability. Not every threat materializes. A tornado is a threat to a data center — but only if the data center is in a tornado-prone region with insufficient protections.
- **Vulnerability -** A vulnerability is a weakness that can be exploited. An unpatched operating system is a vulnerability. An S3 bucket with public read access is a vulnerability. A database password hardcoded in source code on GitHub is a vulnerability.
- **Attack Vector -** An attack vector is a path or method by which an attacker gains access to a system. Attack vectors include network traffic, user input fields, API endpoints, stolen credentials, and even social engineering of employees.
- **Risk** - Risk is the possibility of loss or harm. Risk is a function of the threat, the vulnerability, and the potential impact:
  - Risk = Threat × Vulnerability × Impact
- **Threat Agent -** A threat agent is any entity capable of carrying out an attack. Threat agents include external hackers, disgruntled employees, automated bots, nation-state actors, and even well-meaning developers who accidentally misconfigure a resource.

Security Controls

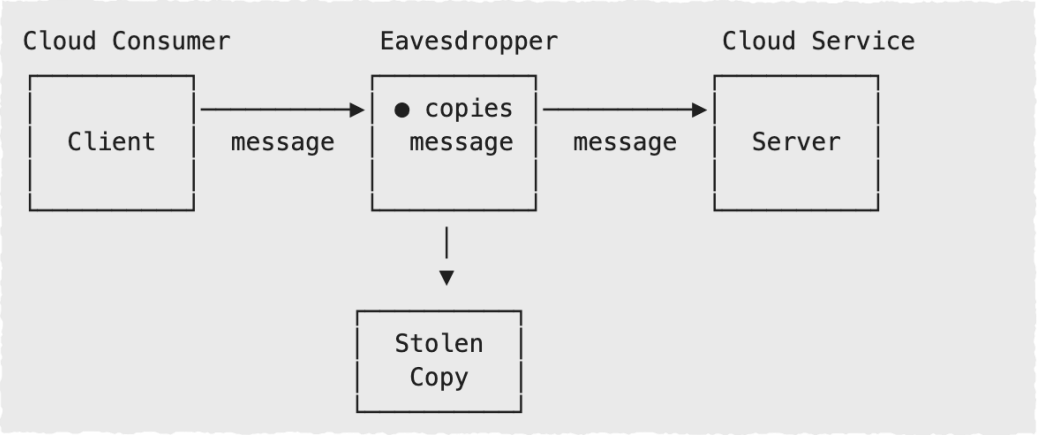
Security controls are countermeasures used to prevent, detect, or respond to security threats. Controls fall into three categories:

- **Preventive** — Stop attacks before they happen (firewalls, encryption, input validation).
- **Detective** — Identify attacks while they are happening or after (logging, monitoring, intrusion detection).
- **Corrective** — Respond to and recover from attacks (backups, incident response plans, auto-remediation).

Cloud Security Threats

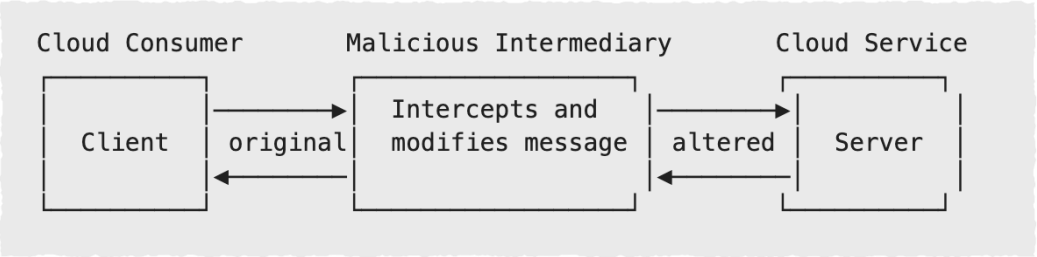
Traffic Eavesdropping

- Traffic eavesdropping occurs when data being transferred to or within a cloud service is passively intercepted by an unauthorized party. The attacker copies data in transit without altering it, making detection extremely difficult.
- This attack targets confidentiality. Because the attacker does not modify the data, the sender and receiver have no indication that interception occurred. Eavesdropping can persist undetected for long periods.
- **Cloud context:** Within a VPC, traffic between instances in the same subnet may travel unencrypted by default. Services like VPC Flow Logs (AWS), VPC Flow Logs (GCP), and NSG Flow Logs (Azure) can detect anomalous traffic patterns but cannot prevent eavesdropping by themselves. Encrypting all traffic — even internal east-west traffic — is the primary defense.



Man-in-the-Middle (Malicious Intermediary)

- A man-in-the-middle (MITM) attack goes beyond passive eavesdropping. The attacker intercepts messages, potentially reads them, alters them, and then forwards the modified message to the destination. This compromises both confidentiality and integrity.
- The attacker may inject malicious data, redirect requests, steal credentials, or modify API responses. TLS with proper certificate validation is the primary defense against MITM attacks.



Insufficient Authorization

- nsufficient authorization occurs when an attacker gains access to resources because permissions were configured too broadly. This is one of the most common cloud security failures.
- Real-world pattern: A developer creates an S3 bucket and sets it to public because they need to share a file quickly. They forget to change it back. Months later, an attacker discovers the bucket and downloads sensitive data. This pattern has caused breaches at multiple major companies.
- **Cloud-specific examples:**
  - An IAM policy with **"Action": "*", "Resource": "*"** (full admin access) attached to an application service account
  - A GCP service account key file committed to a Git repository
  - An Azure storage account with a shared access signature (SAS) token that never expires
  - Security groups allowing inbound traffic on port 22 (SSH) from 0.0.0.0/0 (the entire internet)

Denial of Service (DoS) and Distributed Denial of Service (DDoS)

- A DoS attack overwhelms resources to the point where legitimate users cannot access the service.
- A DDoS attack is the same concept but launched from thousands or millions of compromised machines simultaneously.
- Common DDoS attack types:
  - **Volumetric attacks** flood the network with traffic (e.g., UDP floods, DNS amplification). These are measured in Gbps.
  - **Protocol attacks** exploit weaknesses in network protocols (e.g., SYN floods, Ping of Death). These are measured in packets per second.
  - **Application-layer attacks** target specific application functionality with requests that appear legitimate but are computationally expensive (e.g., complex search queries, large file uploads). These are measured in requests per second and are the hardest to distinguish from real traffic.

Flawed Implementations and Misconfigurations

- Many cloud breaches are not the result of sophisticated hacking — they are the result of misconfiguration. Substandard design, careless defaults, or skipped security reviews create vulnerabilities that attackers discover through automated scanning.
- Common cloud misconfigurations include: publicly accessible databases, overly permissive IAM roles, unencrypted storage, disabled logging, and default credentials left unchanged.

The Shared Responsibility Model

The Core Concept

- Cloud security is a shared responsibility between the cloud provider and the customer. The provider secures the infrastructure that runs the cloud services. The customer secures what they put in the cloud and how they configure it.
- A useful mental model: the provider is responsible for **security OF the cloud**, and the customer is responsible for **security IN the cloud**.

What the Cloud Provider Secures

- The cloud provider is responsible for the physical data centers (guards, biometrics, cameras), the physical servers and networking hardware, the hypervisor and virtualization layer, and the managed service software (e.g., the RDS engine itself, the S3 storage backend).
- You cannot walk into an AWS data center. You cannot access the hypervisor running your EC2 instance. You cannot patch the firmware on the physical network switches. Those are the provider's responsibilities.

What the Customer Secures

The customer is responsible for the guest operating system (including patches and updates), application code and dependencies, IAM configuration (users, roles, policies), network configuration (security groups, NACLs, firewall rules), data encryption choices, and logging and monitoring setup.

Responsibility Shifts by Service Model

- The division of responsibility shifts depending on whether you are using IaaS, PaaS, or SaaS.
- **Key takeaway:** The more managed the service, the less security responsibility falls on you. Using Lambda/Cloud Functions instead of EC2/GCE means you do not need to worry about OS patching — but you are still responsible for your application code, your IAM configuration, and your data.

```
┌──────────────────────────────────────────────────────────────┐
│                   CUSTOMER RESPONSIBILITY                    │
│                                                              │
│  IaaS (EC2/GCE)      PaaS (Lambda/      SaaS (Gmail/        │
│  ──────────────       Cloud Run)         Workspace)         │
│                                                              │
│  ■ OS patching       ■ App code         ■ Data              │
│  ■ App code          ■ Data             ■ Access            │
│  ■ Data              ■ Access             controls          │
│  ■ Network config    ■ Function                             │
│  ■ Firewall rules      config                               │
│  ■ Encryption                                               │
│                                                              │
│──────────────────────────────────────────────────────────────│
│                   PROVIDER RESPONSIBILITY                    │
│                                                              │
│  IaaS                PaaS               SaaS                │
│  ──────────────      ──────────────     ──────────────      │
│                                                              │
│  ■ Hypervisor        ■ OS               ■ OS                │
│  ■ Physical host     ■ Runtime          ■ Runtime           │
│  ■ Network infra     ■ Hypervisor       ■ Network           │
│  ■ Data center       ■ Physical host    ■ App code          │
│                      ■ Network infra    ■ Hypervisor        │
│                      ■ Data center      ■ Physical host     │
│                                         ■ Data center       │
│                                                              │
│  ▲ More customer responsibility  Less customer ▶             │
└──────────────────────────────────────────────────────────────┘
```

Encryption, Hashing & Digital Signatures

Why Encryption Matters in the Cloud

Data transmitted over a network in plaintext is readable by anyone who can intercept it. In the cloud, your data travels over shared physical infrastructure. Even within a VPC, traffic passes through switches and routers managed by the provider. Encryption ensures that intercepted data is useless to an attacker.

Symmetric Encryption

- Symmetric encryption uses the same key for both encryption and decryption. Both the sender and receiver must possess the shared secret key.

```
Plaintext ⟶ [ Encrypt with Key K ] ⟶ Ciphertext ⟶ [ Decrypt with Key K ] ⟶ Plaintext
```

- Symmetric encryption is fast, making it suitable for encrypting large volumes of data. Common algorithms include AES-128, AES-256, and (historically) Triple-DES.
- **The key distribution problem:** If two parties need the same key, how do they securely share it in the first place? If they send the key over the network, an eavesdropper could intercept it. This is where asymmetric encryption comes in.
- **Non-repudiation limitation:** Because multiple parties share the same key, you cannot prove which party performed a specific encryption or decryption. If Alice and Bob both have the key, Bob cannot prove that a given ciphertext was created by Alice and not by himself.

Asymmetric Encryption

- Asymmetric encryption (public-key cryptography) uses two mathematically related keys: a private key known only to the owner, and a public key shared openly.

```
Encrypt with Public Key  ⟶  Only Private Key can decrypt
Encrypt with Private Key ⟶  Only Public Key can decrypt
```

- If Alice wants to send a confidential message to Bob, she encrypts it with Bob's public key. Only Bob's private key can decrypt it. If Bob wants to prove he authored a message, he encrypts (signs) it with his private key. Anyone with his public key can verify it came from him.
- Asymmetric encryption is significantly slower than symmetric encryption (often 100–1000x slower), which is why it is not used to encrypt bulk data directly.
- Common algorithms include RSA (most widely used), ECDSA, and Ed25519.

How TLS Combines Both: The Handshake

- TLS (Transport Layer Security) solves the key distribution problem by using asymmetric encryption to securely exchange a symmetric key, then switching to symmetric encryption for the actual data transfer.
- Every API endpoint you deploy should be served over HTTPS. HTTP without TLS means every request and response — including authentication tokens — is transmitted in plaintext.

```
Client                                                  Server
  |                                                       |
  |——— 1. CLIENT HELLO ———————————————————————————→|
  |     (supported cipher suites, TLS version)            |
  |                                                       |
  |←——— 2. SERVER HELLO ——————————————————————————|
  |     (chosen cipher, server certificate)               |
  |                                                       |
  |     3. Client verifies server certificate             |
  |        against trusted Certificate Authorities        |
  |                                                       |
  |——— 4. PRE-MASTER SECRET ——————————————————————→|
  |     (encrypted with server's public key)              |
  |                                                       |
  |     5. Both sides derive symmetric session key        |
  |        from pre-master secret                         |
  |                                                       |
  |←═══ 6. ENCRYPTED DATA EXCHANGE ═══════════════════→|
  |     (symmetric encryption using session key)          |
  |                                                       |
```

Hashing

- Hashing is a one-way function that converts input data of any size into a fixed-length output (the hash or digest). Hashing is not encryption — you cannot reverse a hash to get the original data.
- Properties of a good hash function: deterministic (same input always produces the same hash), fast to compute, infeasible to reverse, small changes in input produce drastically different output (avalanche effect), and infeasible to find two different inputs that produce the same hash (collision resistance).
- Notice that changing a single character (uppercase W to lowercase w) produces a completely different hash. This is the avalanche effect.
- **Cloud applications of hashing:** Password storage (never store passwords in plaintext — store the hash), data integrity verification (S3 ETags, GCS CRC32C checksums), message authentication codes (HMAC), and Git commit identifiers.

Digital Signatures

- A digital signature combines hashing and asymmetric encryption to provide both integrity and authenticity.
- Digital signatures provide non-repudiation: only the holder of the private key could have created the signature, so the sender cannot deny having sent the message.
- **Cloud example:** When you sign API requests to AWS (Signature Version 4), you are creating a digital signature. AWS verifies that the request came from someone who possesses the secret access key, and that the request was not modified in transit.

```
Sender:
  1. Hash the message              ⟶  Message Digest
  2. Encrypt digest with           ⟶  Digital Signature
     sender's PRIVATE key
  3. Send message + signature

Receiver:
  1. Decrypt signature with        ⟶  Received Digest
     sender's PUBLIC key
  2. Hash the received message     ⟶  Computed Digest
  3. Compare: Received Digest == Computed Digest?
     ✓ Match = message is authentic and unaltered
     ✗ Mismatch = message was tampered with
```

Encryption in the Cloud — KMS, Secrets & Certificates

Encryption at Rest vs. Encryption in Transit

- **Encryption at rest** protects data stored on disk — database records, files in object storage, EBS volumes. If someone gains physical access to the storage device, encrypted data is unreadable.
- **Encryption in transit** protects data moving across networks — API calls, database connections, inter-service communication. TLS is the standard mechanism.
- Both are necessary. Encrypting data in transit but storing it unencrypted on disk still leaves it vulnerable if the storage is compromised.

Cloud Key Management Services

- Encryption keys are the most sensitive secrets in your infrastructure.
- Cloud key management services provide hardware-backed storage and management of encryption keys.

Feature	AWS KMS	GCP Cloud KMS	Azure Key Vault
Key storage	AWS-managed HSMs	Google-managed HSMs	Azure-managed HSMs
Customer-managed keys (CMK)	Yes	Yes (CMEK)	Yes
Automatic key rotation	Annual (configurable)	Configurable period	Configurable period
Integration with storage	S3, EBS, RDS, DynamoDB, etc.	GCS, Persistent Disk, Cloud SQL, BigQuery, etc.	Blob Storage, Managed Disks, Azure SQL, etc.
Bring Your Own Key (BYOK)	Yes	Yes	Yes
Pricing model	Per key + per API call	Per key version + per operation	Per key + per operation

KMS - Default encryption behavior

Service Type	AWS	GCP	Azure
Object storage	S3 encrypts all objects by default (SSE-S3)	GCS encrypts all objects by default (Google-managed keys)	Blob Storage encrypts by default (Microsoft-managed keys)
Block storage	EBS encrypted by default (opt-in at account level)	Persistent Disks encrypted by default	Managed Disks encrypted by default
Managed databases	RDS supports encryption at rest (must enable at creation)	Cloud SQL encrypted by default	Azure SQL TDE enabled by default

Secrets Management -Stop Hardcoding Credentials

- One of the most dangerous and common mistakes in cloud development: hardcoding secrets (database passwords, API keys, tokens) directly in application source code.
- **Why this is catastrophic:** Source code is stored in version control. Even if you delete the secret later, it remains in the Git history. If the repository is public (or becomes public accidentally), the secrets are exposed. Automated bots scan GitHub continuously for exposed AWS keys and can exploit them within minutes.
- **The correct approach:** Use a dedicated secrets management service.

Secrets Management

Feature	AWS Secrets Manager	GCP Secret Manager	Azure Key Vault (Secrets)
Automatic rotation	Yes (built-in for RDS, Redshift, DocumentDB)	No built-in rotation (use Cloud Functions)	Yes (for supported services)
Versioning	Yes	Yes	Yes
Access control	IAM policies	IAM policies	Azure RBAC + access policies
Audit logging	CloudTrail	Cloud Audit Logs	Azure Monitor
Pricing	~$0.40/secret/month + $0.05 per 10K API calls	Free for first 6 active versions; $0.06/version/month + $0.03 per 10K operations	$0.03 per 10K operations

Certificate Management

TLS certificates are required for HTTPS. Cloud providers offer managed certificate services that automatically provision and renew certificates, eliminating the risk of expired certificates causing outages.

Feature	AWS Certificate Manager (ACM)	GCP Certificate Manager	Azure App Service Certificates
Free public certificates	Yes (for use with AWS services)	Yes (for use with GCP load balancers)	No (must purchase)
Auto-renewal	Yes	Yes	Yes (for managed certs)
Integrates with	ALB, CloudFront, API Gateway	HTTPS Load Balancers, Cloud CDN	App Service, Application Gateway, Front Door
DNS validation	Yes	Yes	Yes
Wildcard certs	Yes	Yes	Yes

Cloud Network Security

Layered Network Security

```
Internet
    │
    ▼
┌──────────────────────────┐
│  DDoS Protection         │  ◄── Shield / Cloud Armor / DDoS Protection
│  (Layer 3/4)             │
└──────────────────────────┘
              │
              ▼
┌──────────────────────────┐
│  Web Application Firewall│  ◄── WAF / Cloud Armor Rules / Azure WAF
│  (Layer 7)               │
└──────────────────────────┘
              │
              ▼
┌──────────────────────────┐
│  Load Balancer           │  ◄── ALB / GCP LB / Azure App Gateway
│  (TLS termination)       │
└──────────────────────────┘
              │
              ▼
┌──────────────────────────┐
│  Subnet-level firewall   │  ◄── NACLs / GCP Firewall Rules / NSGs
│  (stateless or stateful) │
└──────────────────────────┘
              │
              ▼
┌──────────────────────────┐
│  Instance-level firewall │  ◄── Security Groups / GCP Firewall / NSGs
│  (stateful)              │
└──────────────────────────┘
              │
              ▼
┌──────────────────────────┐
│  Application             │  ◄── Your code, input validation, AuthN/AuthZ
└──────────────────────────┘
```

Cloud Firewall Comparison

Feature	AWS Security Groups	AWS NACLs	GCP Firewall Rules	Azure NSGs
Scope	Instance (ENI) level	Subnet level	VPC network level	Subnet or NIC level
Stateful/Stateless	Stateful	Stateless	Stateful	Stateful
Default behavior	Deny all inbound, allow all outbound	Allow all inbound and outbound	Deny all inbound, allow all outbound (implied rules)	Deny all inbound, allow VNet and LB inbound
Rule type	Allow only	Allow and Deny	Allow and Deny	Allow and Deny
Priority-based	No (all rules evaluated)	Yes (numbered rules, lowest first)	Yes (priority value, lowest number = highest priority)	Yes (priority value)

Common dangerous mistake: Setting inbound rules to allow `0.0.0.0/0` on port 22 (SSH) or port 3389 (RDP). This opens your instance to login attempts from the entire internet. Automated bots scan for this constantly.

Web Application Firewall (WAF)

A WAF inspects HTTP/HTTPS traffic and blocks requests that match known attack patterns (SQL injection, XSS, etc.). It operates at Layer 7 (application layer) and is typically attached to a load balancer or CDN.

Feature	AWS WAF	GCP Cloud Armor	Azure WAF
Attaches to	ALB, CloudFront, API Gateway	Global/Regional HTTP(S) LB	Application Gateway, Front Door, CDN
Managed rule sets	AWS Managed Rules, Marketplace rules	Preconfigured WAF rules (OWASP Top 10)	OWASP 3.x rule sets (built-in)
Custom rules	Yes	Yes	Yes
Rate limiting	Yes	Yes	Yes
Bot control	AWS WAF Bot Control (add-on)	reCAPTCHA Enterprise integration	Azure Bot Manager
Geographic blocking	Yes	Yes	Yes

Application Security — Input Validation & Common Attacks

The Fundamental Problem

- The core security problem in any web application: **users can submit arbitrary input**. Every piece of data sent from the client to the server — URL parameters, form fields, HTTP headers, cookies, file uploads — must be treated as potentially malicious.
- Client-side validation (JavaScript in the browser) is a user experience feature, not a security feature. An attacker can bypass all client-side validation by sending HTTP requests directly using tools like curl, Postman, or Burp Suite.

SQL Injection

- SQL injection occurs when user-supplied input is concatenated directly into a SQL query without sanitization, allowing the attacker to inject their own SQL commands.
- Vulnerable code pattern (pseudocode):

```
query = "SELECT * FROM users WHERE name = '" + userName + "';"
```

- If userName is set to ' OR '1'='1, the query becomes:

```
SELECT * FROM users WHERE name = '' OR '1'='1';
```

- This returns every row in the users table because '1'='1' is always true.
- **Defense — Parameterized queries (prepared statements):**

```
query = "SELECT * FROM users WHERE name = ?;"
execute(query, [userName])
```

- With parameterized queries, the database engine treats the user input strictly as data, never as SQL syntax. This is the primary and most effective defense against SQL injection.
- **Cloud-managed database relevance:** Using RDS, Cloud SQL, or Azure SQL does not protect you from SQL injection. The managed service handles server patching and backups, but your application code still constructs the queries. SQL injection is an application-level vulnerability, not an infrastructure-level one.

OWASP Top 10 — The Full Picture

- **Broken Access Control** — Users can act outside their intended permissions
- **Cryptographic Failures** — Sensitive data exposed due to weak or missing encryption
- **Injection** — SQL, NoSQL, OS command, LDAP injection via untrusted input
- **Insecure Design** — Flawed architecture that cannot be fixed by implementation alone
- **Security Misconfiguration** — Default configs, open cloud storage, unnecessary features enabled
- **Vulnerable and Outdated Components** — Using libraries with known vulnerabilities
- **Identification and Authentication Failures** — Weak passwords, missing MFA, session flaws
- **Software and Data Integrity Failures** — Insecure CI/CD pipelines, untrusted deserialization
- **Security Logging and Monitoring Failures** — No audit trail, no alerting on suspicious activity
- **Server-Side Request Forgery (SSRF)** — Application fetches remote resources without validating the URL

Cloud-Native Security Services & Audit Logging

Threat Detection and Security Monitoring

Feature	AWS	GCP	Azure
Threat detection	GuardDuty	Security Command Center (SCC) Premium	Microsoft Defender for Cloud
What it monitors	VPC Flow Logs, DNS logs, CloudTrail, S3 data events, EKS audit logs	Asset inventory, vulnerability findings, threat detection, compliance	VMs, containers, SQL, storage, Key Vault, DNS, network
Intelligence sources	AWS threat intelligence feeds, ML-based anomaly detection	Google threat intelligence, built-in ML	Microsoft threat intelligence, ML
Findings/Alerts	Findings with severity levels in GuardDuty console	Findings in SCC dashboard	Security alerts in Defender dashboard
Cost model	Per volume of data analyzed	Per-resource pricing (SCC Premium)	Per-resource, per-plan pricing

Audit Logging

Feature	AWS CloudTrail	GCP Cloud Audit Logs	Azure Activity Log / Monitor
What it logs	API calls (who, what, when, from where)	Admin Activity, Data Access, System Events	Control plane operations, resource changes
Default behavior	Enabled (90-day history, management events only)	Admin Activity enabled by default; Data Access must be enabled	Activity Log retained 90 days by default
Long-term storage	Deliver to S3 bucket	Route to Cloud Storage or BigQuery	Route to Log Analytics workspace or Storage Account
Immutability	CloudTrail Lake, S3 Object Lock	BigQuery with retention policies	Immutable storage in Storage Account
Cross-account	Organization Trail	Organization-level sinks	Diagnostic Settings across subscriptions

Hardened Server Images

- Hardening is the process of stripping unnecessary software, services, and access from a system to reduce its attack surface.
- Hardening practices include removing unnecessary packages and services, closing unused ports, disabling root SSH access, disabling password-based SSH (use key pairs only), applying the latest security patches, and configuring host-based firewalls.
- Cloud providers offer pre-hardened base images: AWS provides Amazon Linux 2023 (hardened by default), GCP provides COS (Container-Optimized OS) for running containers with a minimal attack surface, and Azure provides Confidential VM images. For custom needs, tools like Packer allow you to build and version your own hardened machine images as part of your CI/CD pipeline.

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS